Lecture 18

MCMC Convergence Diagnostics

Arnab Hazra

(Edited from the lecture notes by Brian J. Reich, NC State)

Recap: Gibbs sampling

A Set initial value $\boldsymbol{\theta}^{(0)} = (\theta_1^{(0)}, ..., \theta_p^{(0)})$

B For iteration t,

FC1 Draw $\theta_1^{(t)} | \{\theta_2^{(t-1)}, ..., \theta_p^{(t-1)}, \mathbf{Y}\}$ from the full conditional posterior $f(\theta_1 | rest)$

FC2 Draw $\theta_2^{(t)} | \{\theta_1^{(t)}, \theta_3^{(t-1)}, ..., \theta_p^{(t-1)}, \mathbf{Y}\}$ from the full conditional posterior $f(\theta_2 | rest)$

...

FCp Draw $\theta_p^{(t)} | \{\theta_1^{(t)}, ..., \theta_{p-1}^{(t)}, \mathbf{Y}\}$ from the full conditional posterior $f(\theta_p | rest)$

We repeat step B S times and get the posterior draws

$$\boldsymbol{\theta}^{(1)}, ..., \boldsymbol{\theta}^{(S)}.$$

Recap: Metropolis-Hastings sampling

- MH sampling is a version of rejection sampling.
- Let θ_j^* be the current value of the parameter being updated and $\theta_{(j)}$ be the current value of all other parameters.
- You propose a random candidate based on the current value, $\theta_j^c \sim q(\theta_j|\theta_j^*)$.
- The candidate is accepted with probability

$$R = \min\left\{1, \frac{p(\theta_j^c|\theta_{(j)}, \mathbf{Y})q(\theta_j^*|\theta_j^c)}{p(\theta_j^*|\theta_{(j)}, \mathbf{Y})q(\theta_j^c|\theta_j^*)}\right\}.$$

- If the candidate is not accepted then you simply retain the previous value and move to the next step.

Recap: Why does MCMC work?

- $\theta^{(0)}$ is not a sample from the posterior and it is an arbitrarily chosen initial value.
- $\theta^{(1)}$ is also unlikely from the posterior. Its distribution depends on $\theta^{(0)}$.
- $\theta^{(2)}$ is also unlikely from the posterior. Its distribution depends on $\theta^{(0)}$ and $\theta^{(1)}$.
- **Theorem**: For any initial values, the chain will eventually converge to the posterior.
- **Theorem**: If $\theta^{(s)}$ is a sample from the posterior, then $\theta^{(s+1)}$ is too.

Tuning the MCMC algoritm

- MCMC is beautiful because it can handle virtually any statistical model and it is usually pretty easy to code.
- However, for hard problems great care must be taken to ensure that the algorithm has converged.
- There are three main decisions:
  - Selecting the initial values
  - Determining if/when the chain(s) has converged
  - Selecting the number of samples needed to approximate the posterior

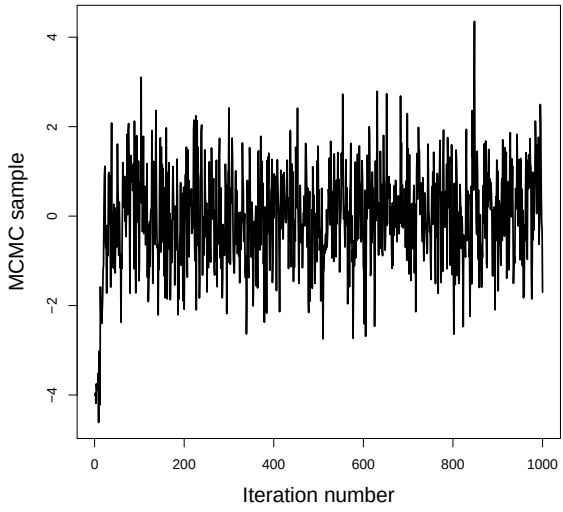
Initial values

- The algorithm will eventually converge no matter what initial values you select.
- However taking time to select good initial values will speed up convergence.
- It is important to try a few initial values to verify they all give the same result.
- Usually 3-5 separate chains is sufficient.
- **Option 1**: Select good initial values using method of moments or MLE.
- **Option 2**: Purposely pick bad but different initial values for each chain to check convergence.

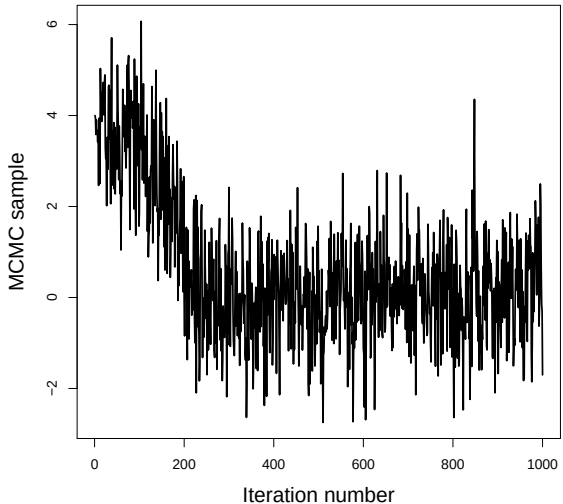
Convergence

- It can take hundreds or even thousands of iterations to move from the initial values to the posterior.
- When the sampler reaches the posterior this is called convergence.
- Samples before convergence are discard as **burn-in**.
- After convergence, the samples should not converge to a single point!
- They should be draws from the posterior, and ideally look like a caterpillar.

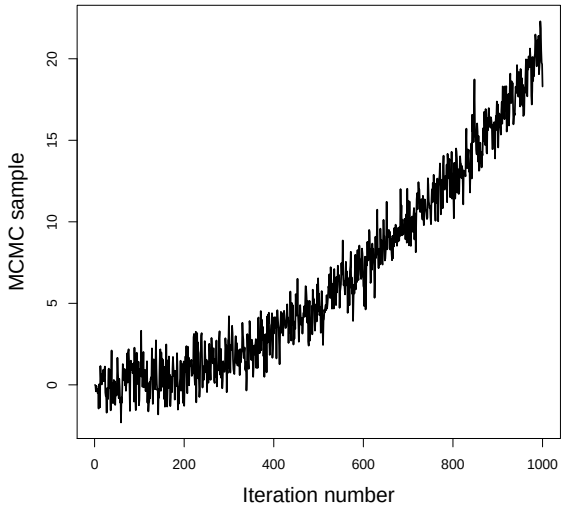
Convergence in a few iterations



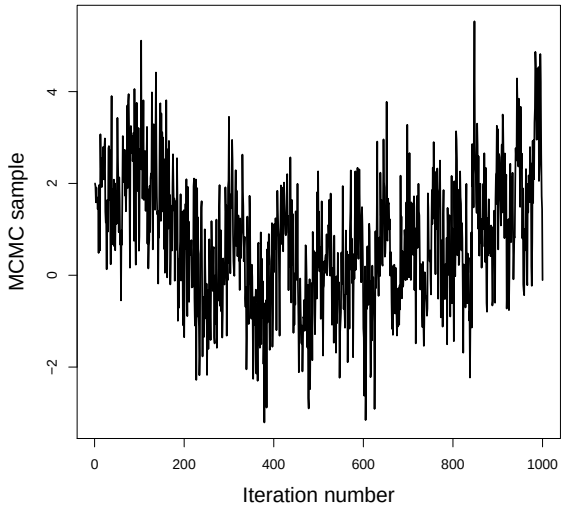
Convergence in a few hundred iterations



This one never converged



Convergence is questionable



Convergence diagnostics

- So far we have visually inspected the chains for convergence.
- There are many formal diagnostics.
- The `CODA` package in `R` has dozens of diagnostics.
- Most give a measure of convergence for each parameter.
- Checking convergence using these one-number summaries is more efficient and objective than visual inspection.

Convergence diagnostics

- Did my chains converge?
  - Geweke
  - Gelman-Rubin
- Did I run the sampler long enough after convergence?
  - Effective sample size
  - Standard errors for the posterior mean estimate

Examples

- The JAGS function `coda.samples` returns sample is the format that can be passed to the `CODA` function which actually computes the diagnostics.

- We will use `CODA` to access convergence for a best-case and a worst-case scenario.

Geweke diagnostic

- It compares the mean in the beginning of the chain (after burn-in period) with the mean at the end of the chain.
- This can be used for a single chain.
- This is done separately for each parameter.
- The JAGS default is to compare the first 10% with the last 50%.
- The test accounts for the inherent autocorrelation in MCMC.
- The test statistic is a z-score, so $|Z| > 2$ indicates poor convergence.

Gelman-Rubin statistic

- If we run multiple chains, we hope that all chains give same result.
- The Gelman-Rubin statistics measures agreement between chains.
- Is it essentially an ANOVA test of whether the chains have the same mean.
- It is scaled so that 1 is perfect and 1.1 is decent but not great convergence.
- JAGS plots the statistic over iteration.
- When the statistic reaches one, this indicates convergence.

Statistical Science
2021, Vol. 36, No. 4, 518–529
https://doi.org/10.1214/20-STS812

Revisiting the Gelman–Rubin Diagnostic

Dootika Vats and Christina Knudson

Abstract. Gelman and Rubin's (*Statist. Sci.* **7** (1992) 457–472) convergence diagnostic is one of the most popular methods for terminating a Markov chain Monte Carlo (MCMC) sampler. Since the seminal paper, researchers have developed sophisticated methods for estimating variance of Monte Carlo averages. We show that these estimators find immediate use in the Gelman–Rubin statistic, a connection not previously established in the literature. We incorporate these estimators to upgrade both the univariate and multivariate Gelman–Rubin statistics, leading to improved stability in MCMC termination time. An immediate advantage is that our new Gelman–Rubin statistic can be calculated for a single chain. In addition, we establish a one-to-one relationship between the Gelman–Rubin statistic and effective sample size. Leveraging this relationship, we develop a principled termination criterion for the Gelman–Rubin statistic. Finally, we demonstrate the utility of our improved diagnostic via examples.

Autocorrelation

- Ideally the samples should be independent across iteration.
- The autocorrelation function $\rho(h)$ is the correlation between samples h iterations apart.
- JAGS plots the autocorrelation as a function of h.
- Lower values are better, but if the chains are long enough even large values can be OK.
- **Thinning**: If autocorrelation is (close to) zero after lag h, you can thin the samples by h to achieve (approximate) independence.
- This is always less efficient than using all samples, but can save memory.

Effective sample size

- Highly correlated samples have less information than independent samples.
- Say S is the actual number of MCMC samples.
- The **effective samples size** is

$$ESS = \frac{S}{1 + 2\sum_{h=1}^{\infty} \rho(h)}.$$

- The correlated MCMC sample of length S has the same information as ESS independent samples.
- In practice, ESS should be at least a few thousand for all parameters.

Standard errors of posterior mean estimates

- The sample mean of the MCMC draws is an estimate of the posterior mean.
- The standard error of this estimate as another diagnostic.
- Assuming independence the standard error is

$$\text{Naive SE} = \frac{s}{\sqrt{S}},$$

  where s is the sample SD and S is the number of samples.

- A more realistic standard error is

$$\text{Times-series SE} = \frac{s}{\sqrt{ESS}}.$$

Thank you!